

# Arhitectura calculatoarelor

## Curs 4

### 4. Proiectarea procesorului cu un singur ciclu de execuție

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

# Objective

1. Fazele de proiectare ale unui procesor
2. Analiza la nivel RTL
3. Selectarea componentelor
4. Disciplina semnalului de ceas
5. Asamblarea componentelor
6. Identificarea semnalelor de control
7. Implementarea logicii de control

# 4.1 Fazele de proiectare ale unui procesor

## 1. Analiza la nivel RTL a setului de instrucțiuni

- Sunt utile diagramele de procesare (curs anterior);
- Sunt determinate elementele esențiale necesare în calea de date.

## 2. Selectarea componentelor și stabilirea disciplinei semnalului de ceas

- Se identifică elementele de stare (registre, memorii) și se stabilește cum se face scrierea/citirea (pe care front);
- Se determină calea cea mai defavorabilă pentru a stabili perioada/frecvența de ceas.

## 3. Asamblarea componentelor în calea de date

- Se realizează treptat începând cu elementele majore (registre, ALU, memorii);
- Se completează secvența de micro-operații la toate instrucțiunile adăugând conexiunile necesare;
- La destinații cu mai multe surse se introduc multiplexoare sau magistrale (bus-uri).

## 4. Identificarea semnalelor de control în calea de date

- Se stabilesc valorile necesare pentru semnalele de control la fiecare instrucțiune.

## 5. Implementarea logicii de control

- Implementarea unității(lor) de control;
- Variante:
  - Combinațional;
  - Hard-wired secvențial;
  - Micro-programat.

## 4.2 Analiza la nivel RTL

### Selecția setului de instrucțiuni

| Instrucțiune           | RTL                                                                                                | Modificare PC        |
|------------------------|----------------------------------------------------------------------------------------------------|----------------------|
| op \$rd, \$rs, \$rt    | $RF[rd] \leftarrow RF[rs] \text{ op } RF[rt],$<br>$\text{op} \in \{\text{add, sub, or, and}\}$     | $PC \leftarrow PC+4$ |
| ORI \$rt, \$rs, imm    | $RF[rt] \leftarrow RF[rs] \text{ or } Z\_Ext(imm)$                                                 | $PC \leftarrow PC+4$ |
| ADDI \$rt, \$rs, imm   | $RF[rt] \leftarrow RF[rs] + S\_Ext(imm)$                                                           | $PC \leftarrow PC+4$ |
| LW \$rt, offset(\$rs)  | $RF[rt] \leftarrow M[RF[rs]+S\_Ext(offset)]$                                                       | $PC \leftarrow PC+4$ |
| SW \$rt, offset(\$rs)  | $M[RF[rs]+S\_Ext(offset)] \leftarrow RF[rt]$                                                       | $PC \leftarrow PC+4$ |
| BEQ \$rs, \$rt, offset | if $RF[rs]=RF[rt]$ then $PC \leftarrow (PC+4) + S\_Ext(offset) \ll 2$<br>else $PC \leftarrow PC+4$ |                      |
| J addr                 | $PC \leftarrow (PC+4)[31:28] \parallel \text{addr} \parallel 00$                                   |                      |

**Notă:** Nu se va implementa pentru toate instrucțiunile MIPS, ci numai pentru un subset reprezentativ.

## 4.2 Analiza la nivel RTL

### Instrucțiuni de tip R

Asamblare

RTL

op \$rd, \$rs, \$rt

$RF[rd] \leftarrow RF[rs] \text{ op } RF[rt]; PC \leftarrow PC+4$

**Scop:** operație între \$rs și \$rt, rezultatul în \$rd.

|        |          |    |    |    |    |          |
|--------|----------|----|----|----|----|----------|
|        | 6        | 5  | 5  | 5  | 5  | 6        |
| Format | opcode=0 | rs | rt | rd | sa | function |

#### Cod mașină

|     |        |    |    |    |       |        |
|-----|--------|----|----|----|-------|--------|
| ADD | 000000 | rs | rt | rd | 00000 | 100000 |
| SUB | 000000 | rs | rt | rd | 00000 | 100010 |
| OR  | 000000 | rs | rt | rd | 00000 | 100101 |
| AND | 000000 | rs | rt | rd | 00000 | 100100 |

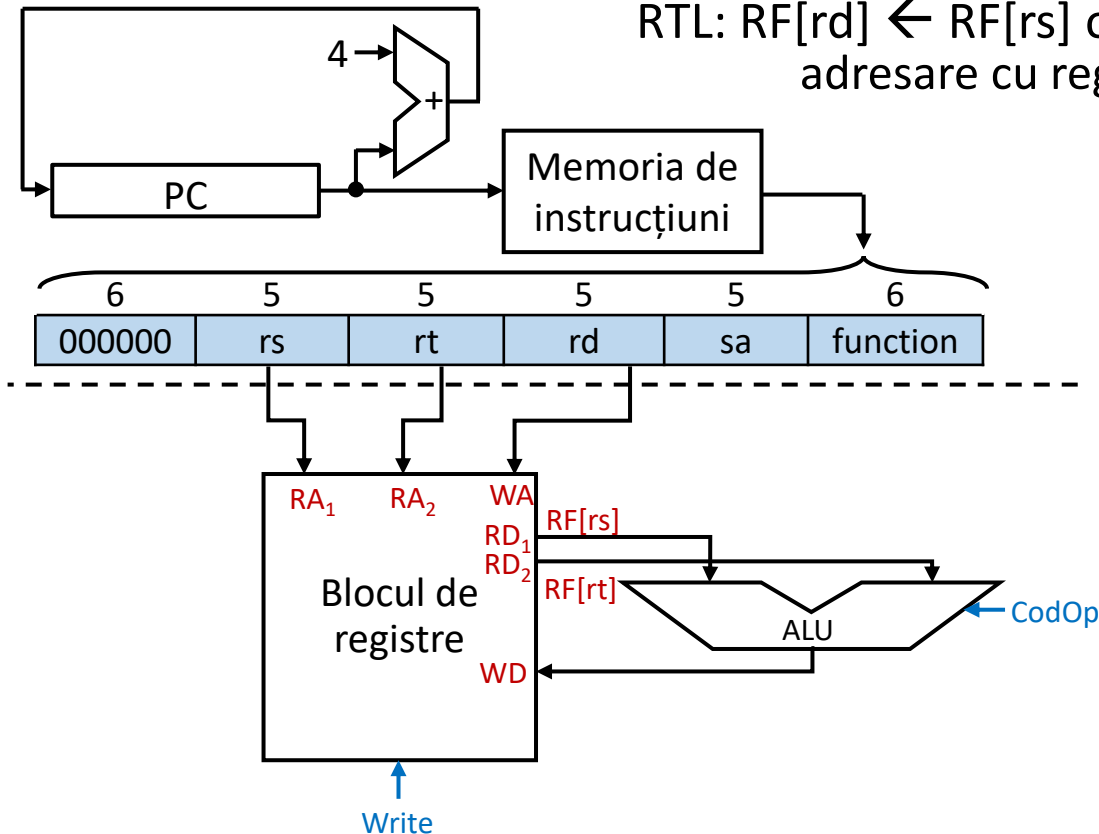
#### Exemple

|                   |        |       |       |       |       |        |
|-------------------|--------|-------|-------|-------|-------|--------|
| ADD \$6, \$4, \$3 | 000000 | 00100 | 00011 | 00110 | 00000 | 100000 |
| OR \$1, \$0, \$7  | 000000 | 00000 | 00111 | 00001 | 00000 | 100101 |

# 4.2 Analiza la nivel RTL

## Instrucțiuni de tip R – diagrama de procesare

RTL:  $RF[rd] \leftarrow RF[rs] \text{ op } RF[rt]; PC \leftarrow PC+4$   
adresare cu registru



| Faza  | Resurse necesare                                                  |
|-------|-------------------------------------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator                              |
| ID/OF | Bloc de registre (OF), <b>Unitatea principală de control (ID)</b> |
| EX    | ALU, <b>Unitatea de control ALU</b>                               |
| MEM   | <b>no operation</b>                                               |
| WB    | Bloc de registre                                                  |

ID = decodificarea instrucțiunii pe baza *opcode* => semnalele de control  
OF = încărcarea operanzilor din registre sau instrucțiune (în cazul imediatului)



**no operation** = operație care nu modifică starea elementului în faza curentă (ex: citire din memorie)

## 4.2 Analiza la nivel RTL

### Instrucțiunea OR Immediate (ORI), de tip I – diagrama de procesare

ORI \$rt, \$rs, imm      RTL:  $RF[rt] \leftarrow RF[rs] \mid Z\_Ext(imm); PC \leftarrow PC+4$

**Scop:** SAU logic între \$rs și imediat, rezultatul în \$rt.

adresare cu registru și cu imediat

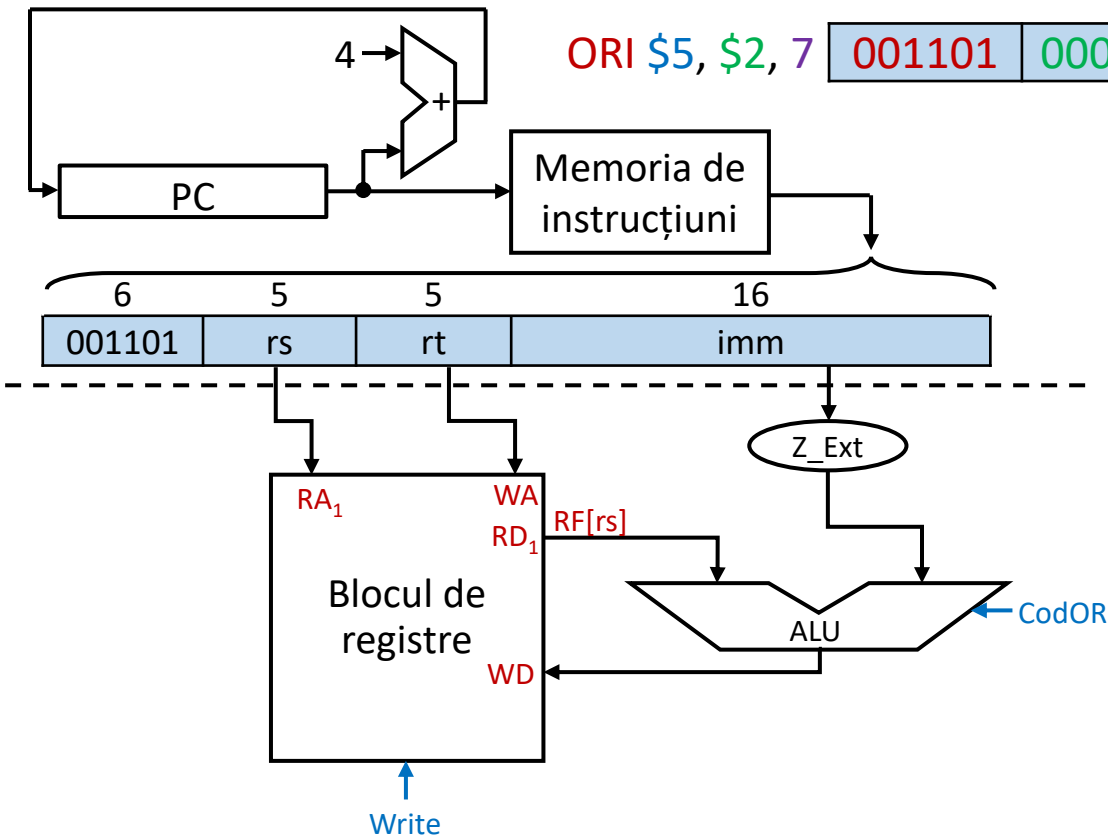
Format

| 6      | 5  | 5  | 16  |
|--------|----|----|-----|
| 001101 | rs | rt | imm |

Exemplu

ORI \$5, \$2, 7

|        |       |       |                  |
|--------|-------|-------|------------------|
| 001101 | 00010 | 00101 | 0000000000000111 |
|--------|-------|-------|------------------|



| Faza  | Resurse necesare                                                   |
|-------|--------------------------------------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator                               |
| ID/OF | Bloc de registre, <u>Extensie</u> , Unitatea principală de control |
| EX    | ALU, Unitatea de control ALU                                       |
| MEM   | no operation                                                       |
| WB    | Bloc de registre                                                   |

# 4.2 Analiza la nivel RTL

## Instrucțiunea ADD Immediate (ADDI), de tip I – diagrama de procesare

ADDI \$rt, \$rs, imm      RTL:  $RF[rt] \leftarrow RF[rs] + S\_Ext(imm); PC \leftarrow PC+4$

**Scop:** adunare între \$rs și imediat, rezultatul în \$rt.

adresare cu registru și cu imediat

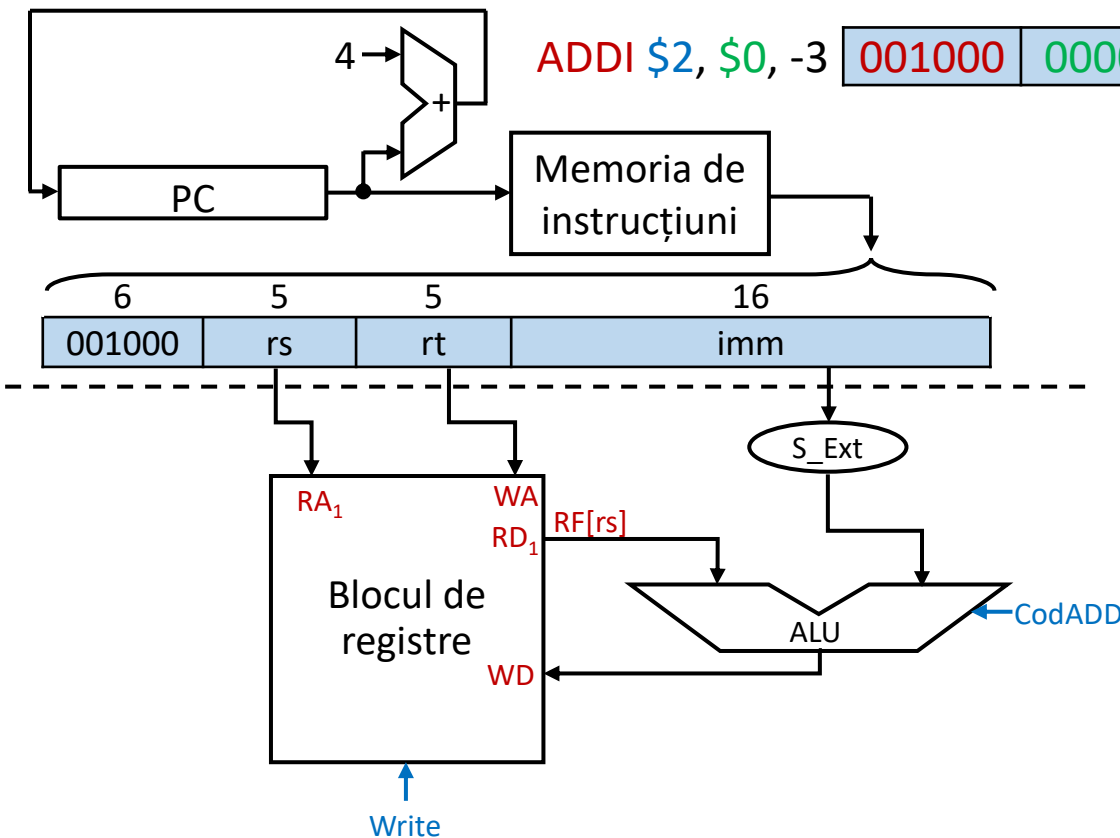
Format

|        |    |    |     |
|--------|----|----|-----|
| 6      | 5  | 5  | 16  |
| 001000 | rs | rt | imm |

Exemplu

ADDI \$2, \$0, -3

|        |       |       |                  |
|--------|-------|-------|------------------|
| 001000 | 00000 | 00010 | 1111111111111101 |
|--------|-------|-------|------------------|



| Faza  | Resurse necesare                                                   |
|-------|--------------------------------------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator                               |
| ID/OF | Bloc de registre, <u>Extensie</u> , Unitatea principală de control |
| EX    | ALU, Unitatea de control ALU                                       |
| MEM   | no operation                                                       |
| WB    | Bloc de registre                                                   |

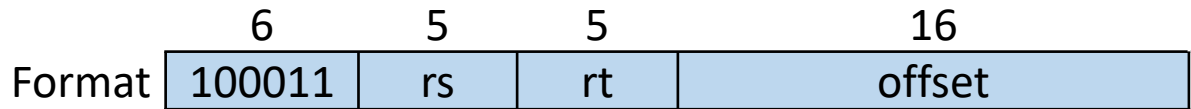
# 4.2 Analiza la nivel RTL

## Instrucțiunea Load Word (LW), de tip I – diagrama de procesare

LW \$rt, offset(\$rs)     RTL:  $RF[rt] \leftarrow M[RF[rs] + S\_Ext(offset)]; PC \leftarrow PC + 4$

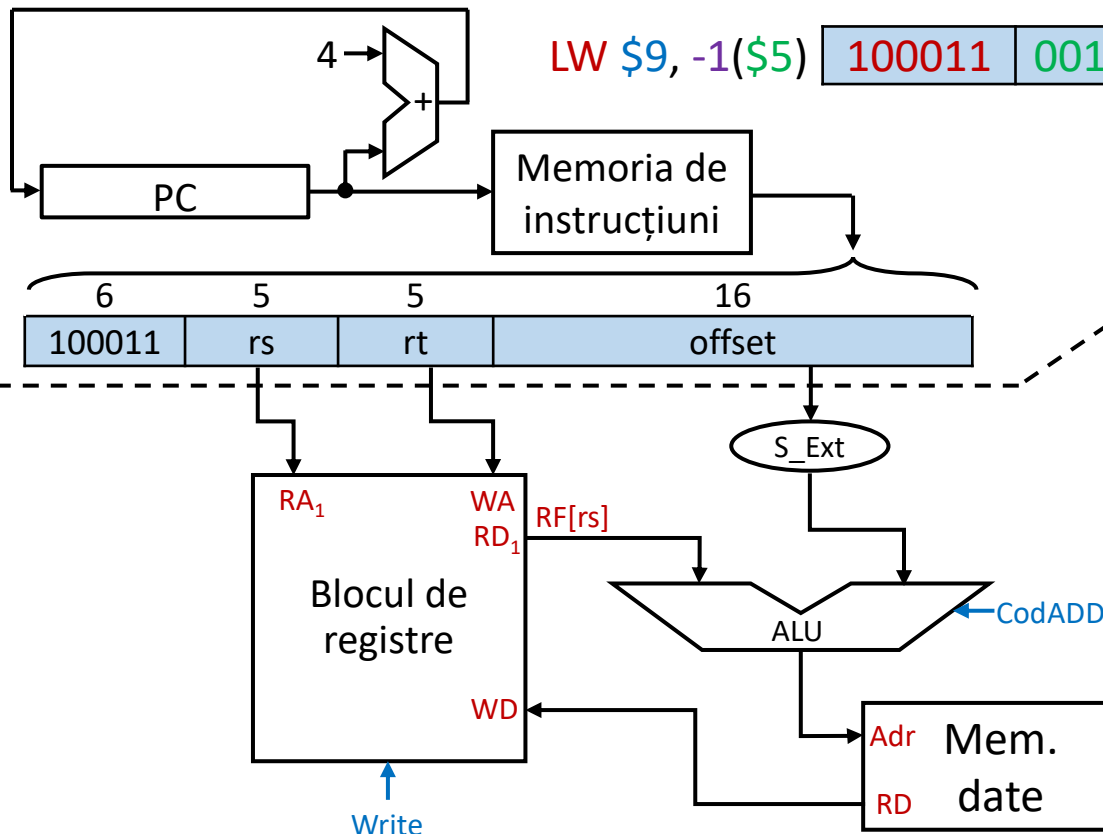
**Scop:** încarcă 32 biți din memoria de date în \$rt.

adresare cu registru și cu bază și index



Exemplu

LW \$9, -1(\$5)



| Faza  | Resurse necesare                                           |
|-------|------------------------------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator                       |
| ID/OF | Bloc de registre, Extensie, Unitatea principală de control |
| EX    | ALU, Unitatea de control ALU                               |
| MEM   | Memoria de date                                            |
| WB    | Bloc de registre                                           |

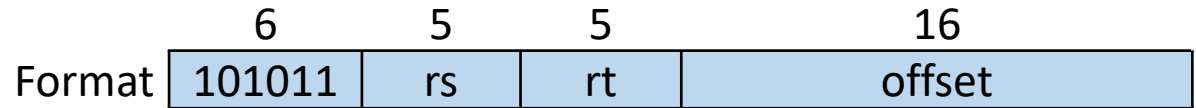
# 4.2 Analiza la nivel RTL

## Instrucțiunea Store Word (SW), de tip I – diagrama de procesare

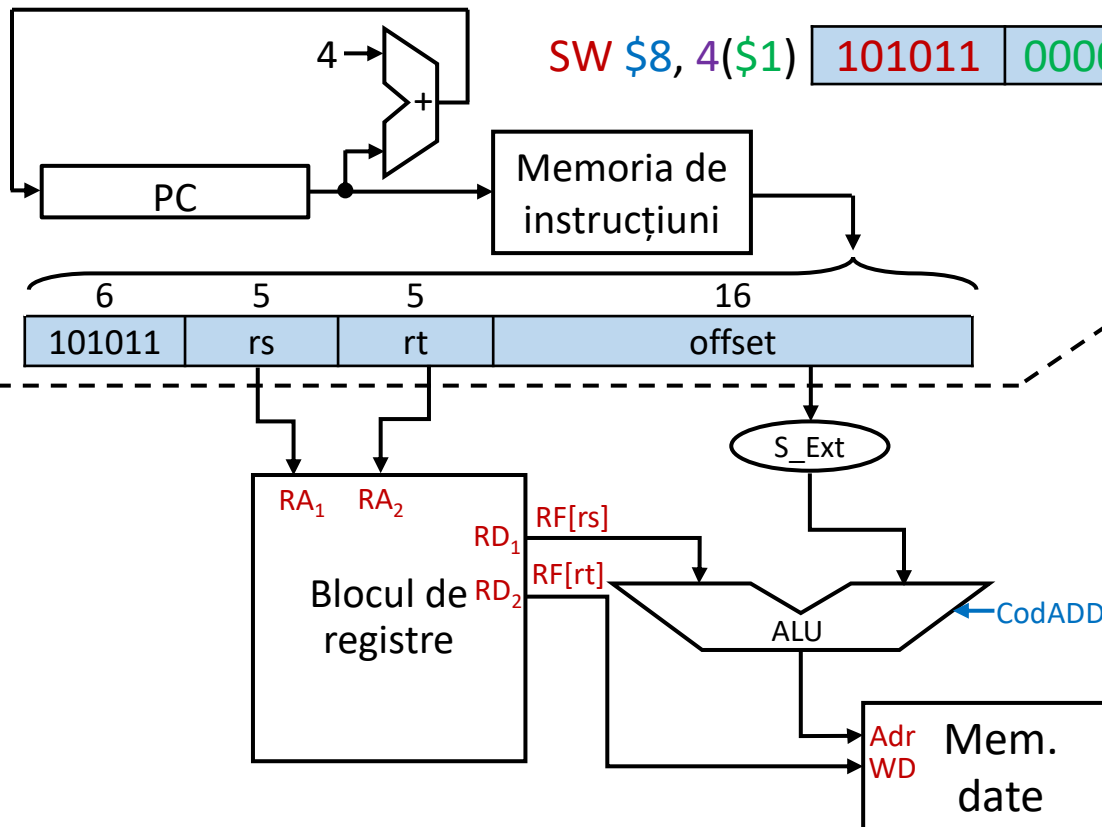
SW \$rt, offset(\$rs)      RTL:  $M[RF[rs] + S\_Ext(offset)] \leftarrow RF[rt]; PC \leftarrow PC + 4$

**Scop:** încarcă 32 biți din \$rt în memoria de date.

adresare cu registru și cu bază și index



Exemplu



| Faza  | Resurse necesare                                           |
|-------|------------------------------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator                       |
| ID/OF | Bloc de registre, Extensie, Unitatea principală de control |
| EX    | ALU, Unitatea de control ALU                               |
| MEM   | Memoria de date                                            |
| WB    | no operation                                               |

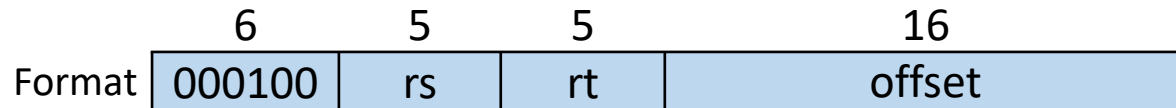
## 4.2 Analiza la nivel RTL

### Instrucțiunea Branch on Equal (BEQ), de tip I – **diagrama de procesare**

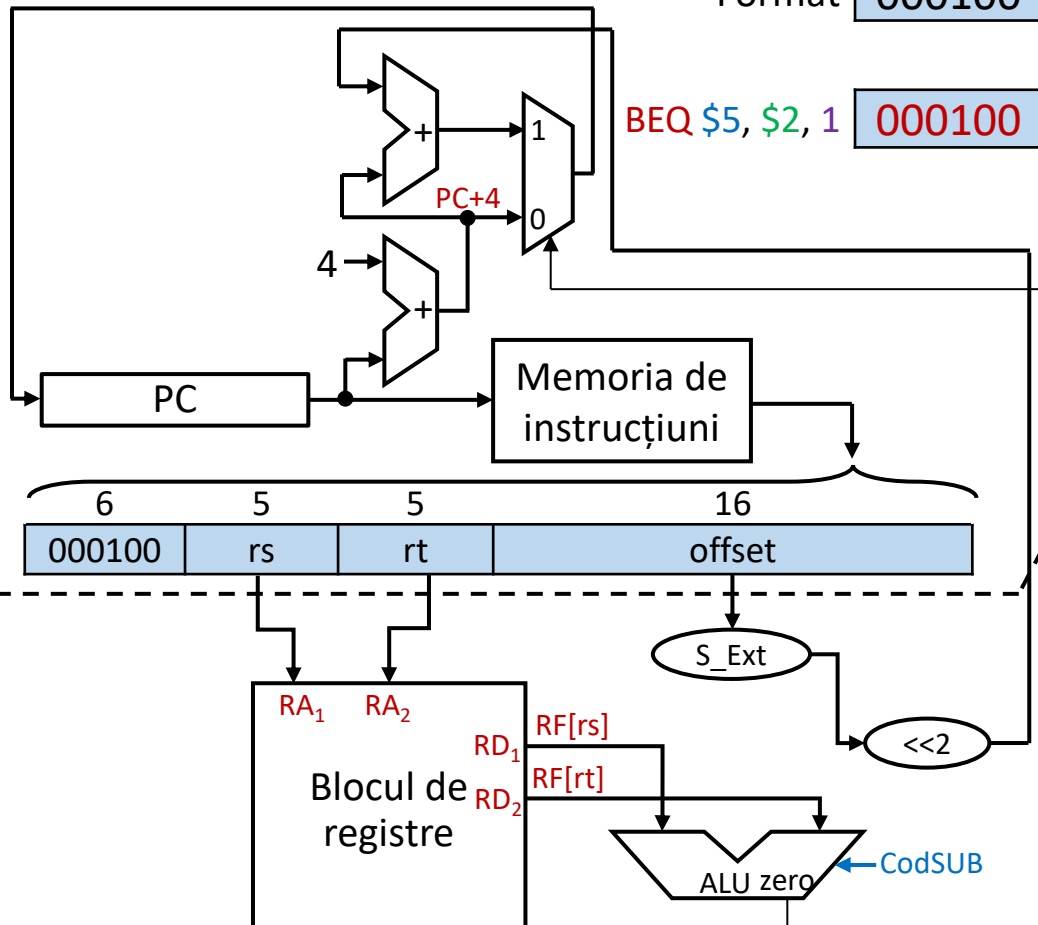
BEQ \$rs, \$rt, offset    RTL: if RF[rs]=RF[rt] then PC  $\leftarrow$  (PC+4) + S\_Ext(offset)<<2

**Scop:** compară 2 registre pentru salt relativ la PC+4.

else PC  $\leftarrow$  PC+4  
adresare relativă la PC+4



Exemplu



| Faza  | Resurse necesare                                           |
|-------|------------------------------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator                       |
| ID/OF | Bloc de registre, Extensie, Unitatea principală de control |
| EX    | ALU, Unitatea de control ALU, Shifter, Sumator, MUX        |
| MEM   | no operation                                               |
| WB    | no operation                                               |

## 4.2 Analiza la nivel RTL

### Instrucțiunea Jump (J), de tip J – diagrama de procesare

J address

RTL:  $PC \leftarrow (PC+4)[31:28] || \text{address} || 00$

adresare pseudo-directă (în segment)

**Scop:** salt necondiționat la adresa specificată.

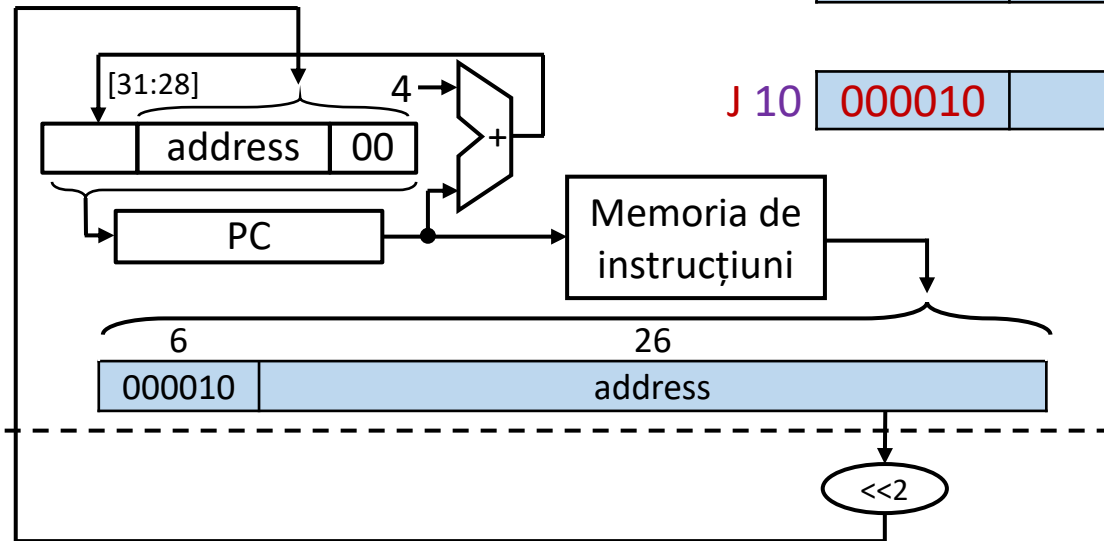
Format

|        |         |
|--------|---------|
| 6      | 26      |
| 000010 | address |

Exemplu

J 10

|        |                              |
|--------|------------------------------|
| 000010 | 0000000000000000000000001010 |
|--------|------------------------------|



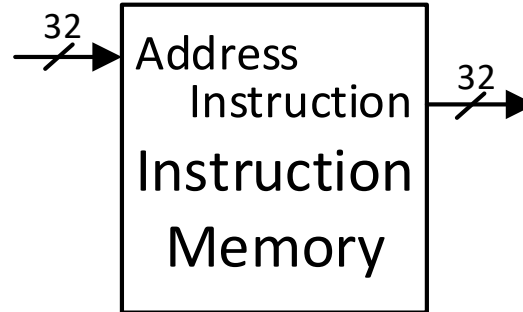
| Faza  | Resurse necesare                     |
|-------|--------------------------------------|
| IF    | PC, Memoria de instrucțiuni, sumator |
| ID/OF | Unitatea principală de control       |
| EX    | Shifter (concatenare 00)             |
| MEM   | no operation                         |
| WB    | no operation                         |

## 4.3 Selectarea componentelor



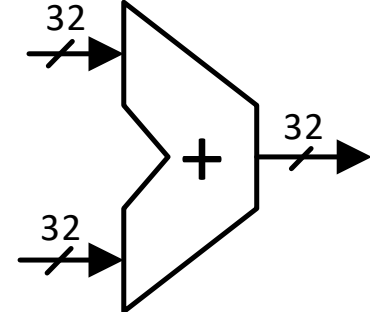
**Contorul de program**  
(Program Counter - PC):

- registru pe 32 de biți.



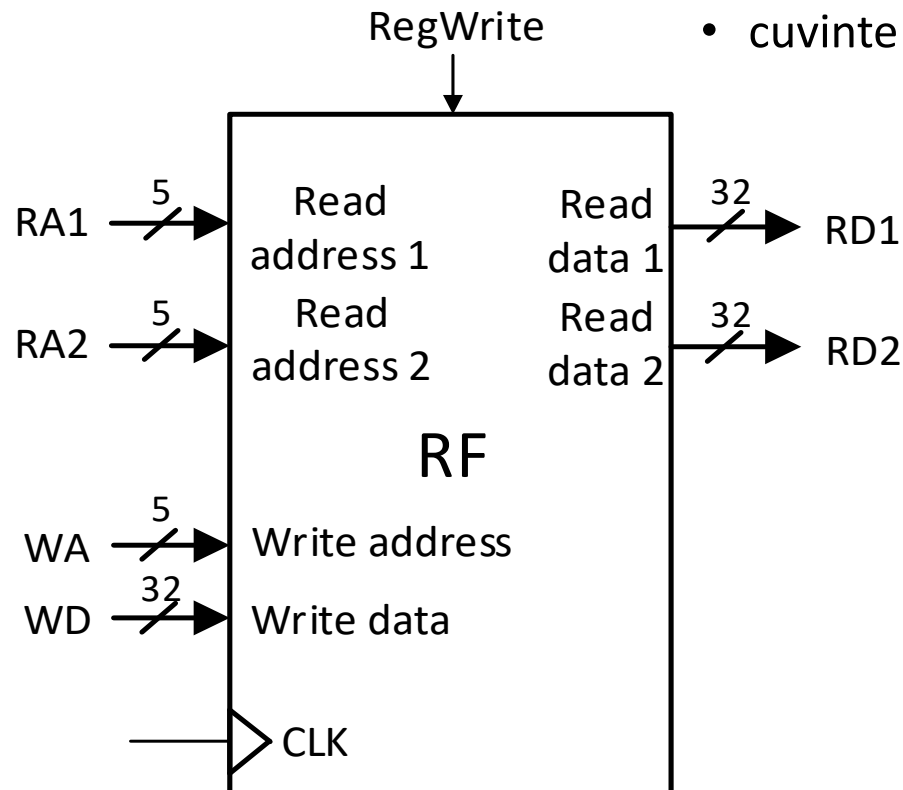
**Memoria de instrucțiuni**  
(memorie ROM):

- cuvinte pe 32 de biți.



**Sumator** pe 32 biți:

- sunt necesare 2 unități pentru calculul PC.

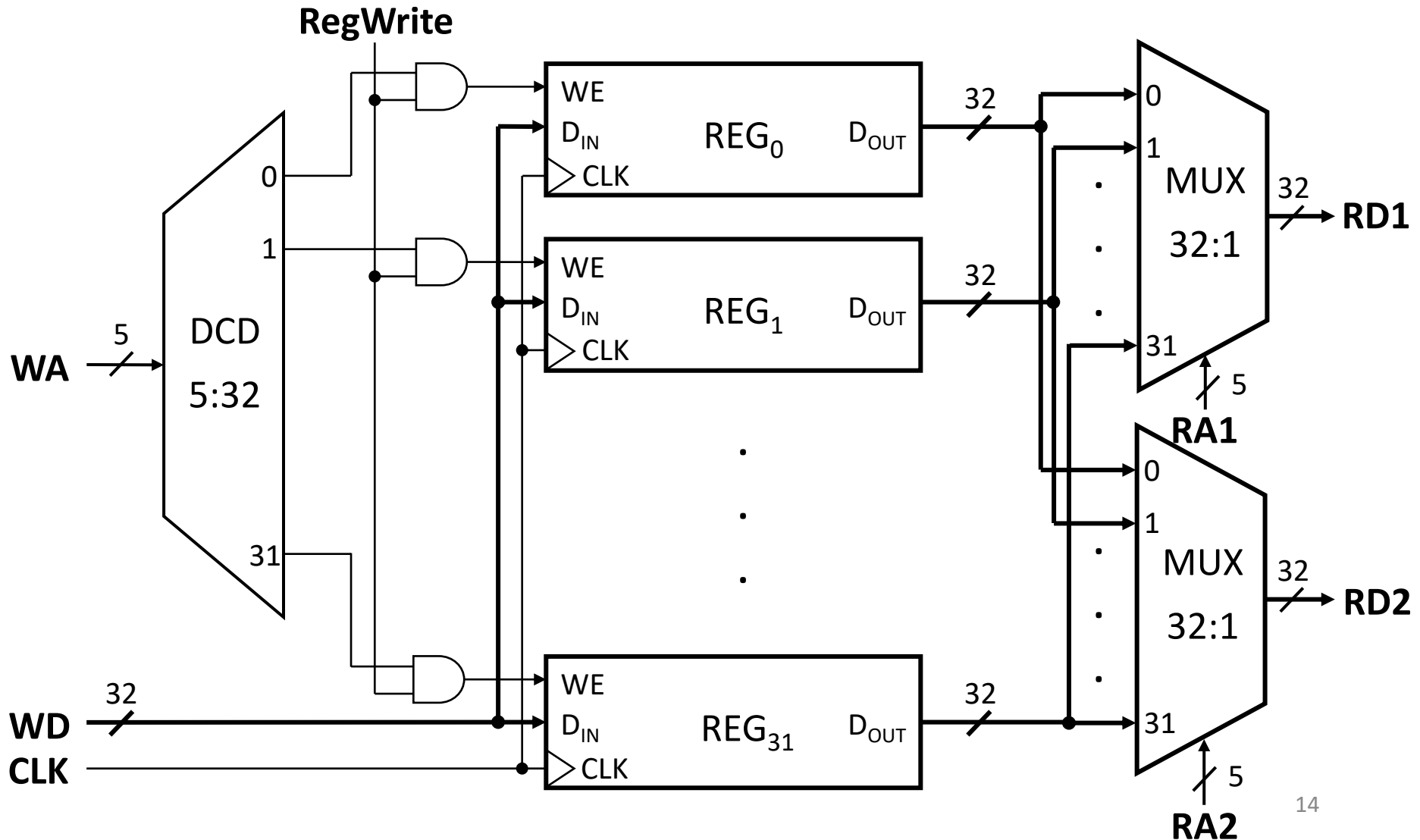


**Blocul de registre** (Register File – RF):

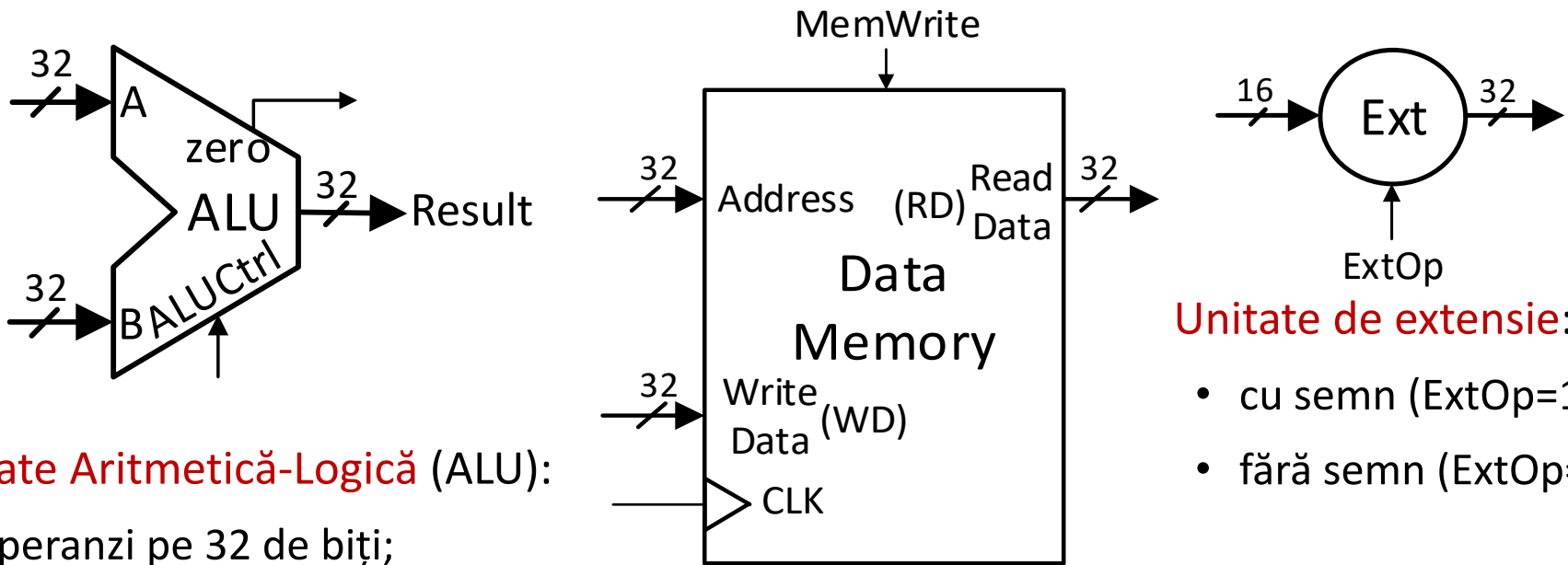
- 32 registre pe 32 de biți;
- 2 porturi de citire (RA1/RD1, RA2/RD2);
- 1 port de scriere (WA/WD);
- Comanda de scriere RegWrite;
- Scriere sincronă pe CLK, dacă RegWrite=1;
- Citire asincronă (răspuns imediat).

## 4.3 Selectarea componentelor

### Blocul de registre (RF) – implementare



## 4.3 Selectarea componentelor



### Unitate Aritmetică-Logică (ALU):

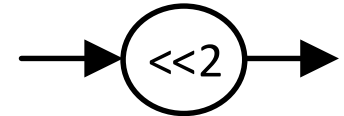
- Operanzi pe 32 de biți;
- Se implementează conform cu operațiile de implementat.
- Prezintă indicatori (flags) suplimentari (ex. rezultat zero).

### Memorie de date (memorie RAM):

- cuvinte pe 32 de biți;
- 1 port de scriere/citire (Address/WD/RD);
- Comanda de scriere MemWrite;
- Scriere sincronă pe CLK, dacă MemWrite=1;
- Citire asincronă (răspuns imediat).

### Unitate de extensie:

- cu semn (ExtOp=1);
- fără semn (ExtOp=0).



### Unitate de deplasare:

- 2 astfel de unități;
  - 1 unitate concatenează 00 pentru Jump;
  - cealaltă, deplasare cu 2 biți.

## 4.4 Disciplina semnalului de ceas

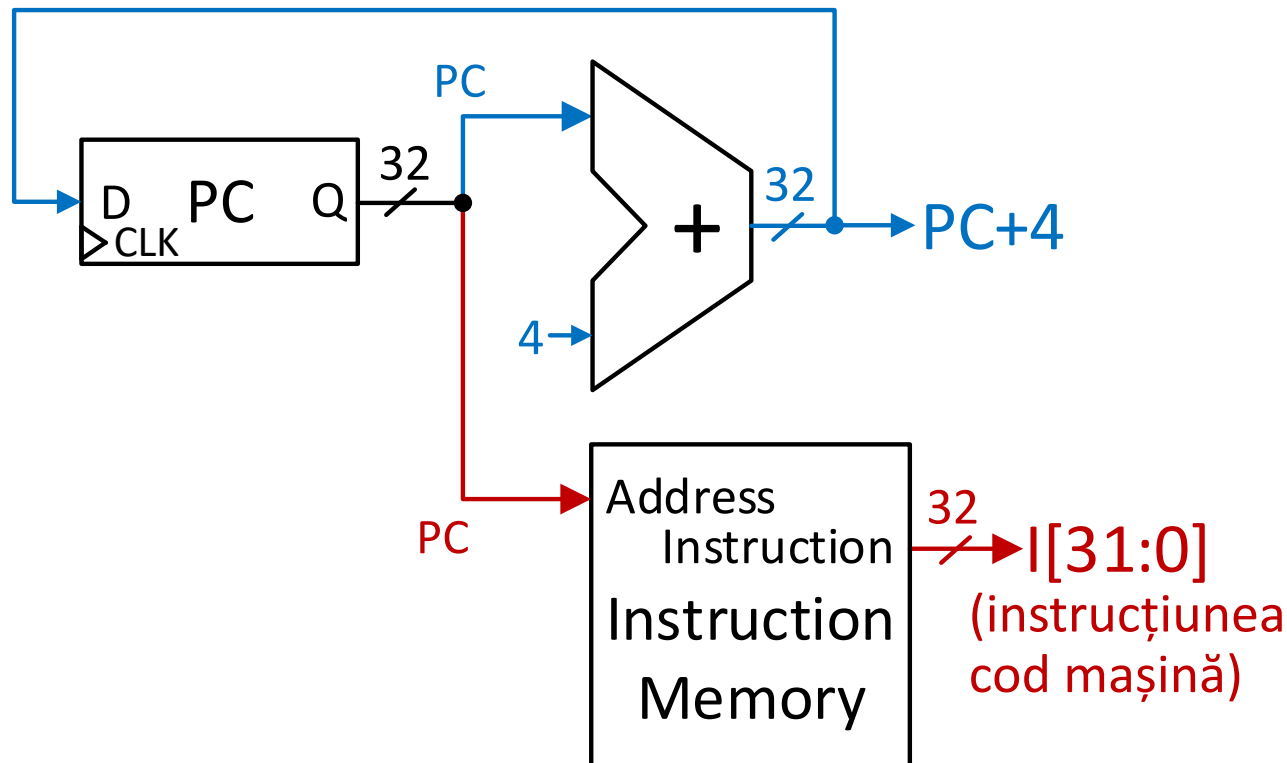
- Scriere pe front:
  - ascendent;
  - descendent;
  - ambele.
- Se stabilește **scrierea pe front ascendent** la:
  - Contorul de program (PC);
  - Blocul de registre (RF);
  - Memoria de date.

**Notă:** La scriere sincronă datele de intrare trebuie să fie menținute stabile cu  $T_{\text{setup}}$  înainte de frontul de ceas și cu  $T_{\text{hold}}$  după front.

## 4.5 Asamblarea componentelor

### Instruction Fetch (IF)

- $I[31:0] \leftarrow IM[PC]; PC \leftarrow PC+4$



## 4.5 Asamblarea componentelor

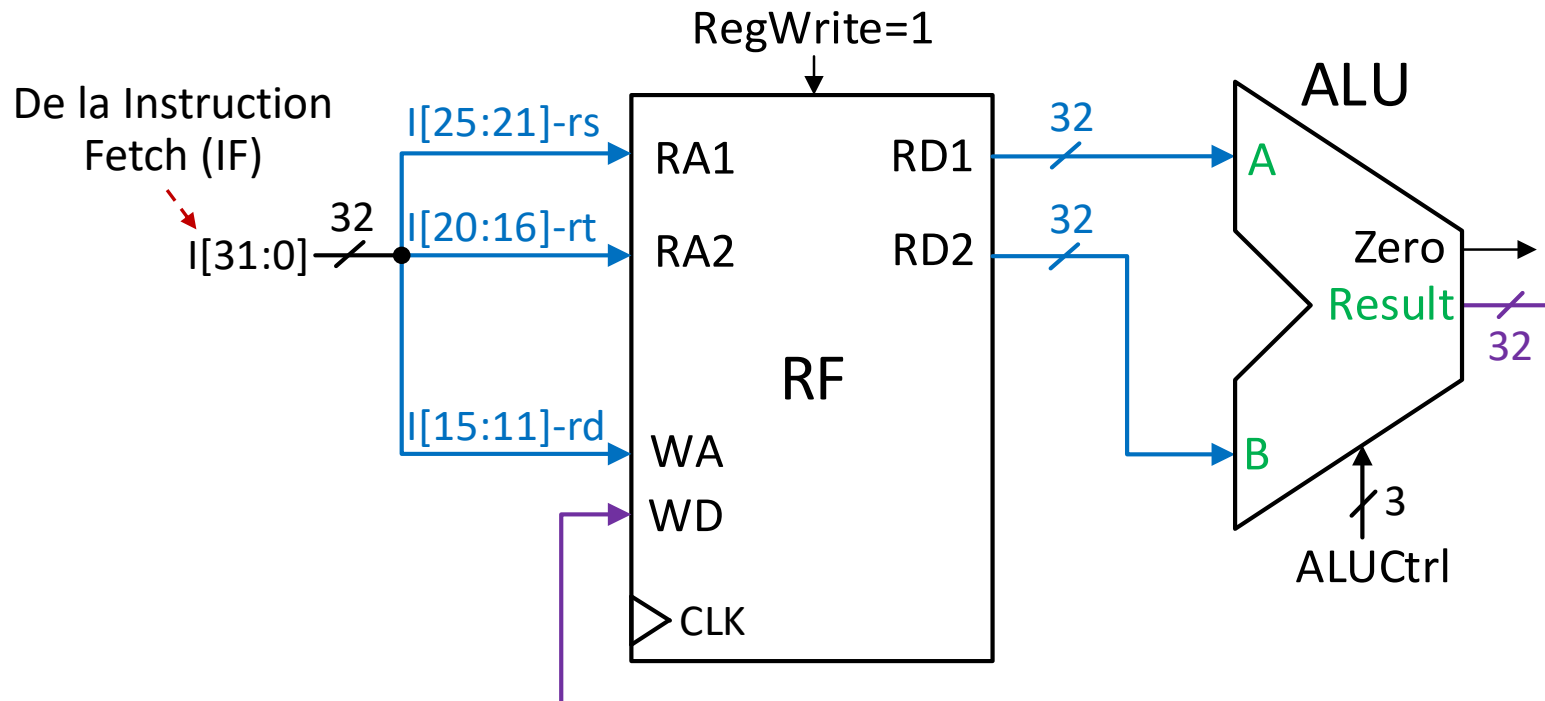
**Tip R:**  $RF[rd] \leftarrow RF[rs] \text{ op } RF[rt]$

Fazele instrucțiunii (*la ciclu unic nu se referă la cicluri de ceas*):

1.    RA1 ← rs; RA2 ← rt; WA ← rd  
     A ← RD1 ← RF[RA1]; B ← RD2 ← RF[RA2]

} Operand Fetch (OF)  
           } încărcarea operanzilor

- 2.    Result ← A op B – Execute (EX), operația indicată de ALU Ctrl
- 3.    RF[WA] ← WD ← Result – Write-Back (WB), datele sunt pregătite pentru scriere

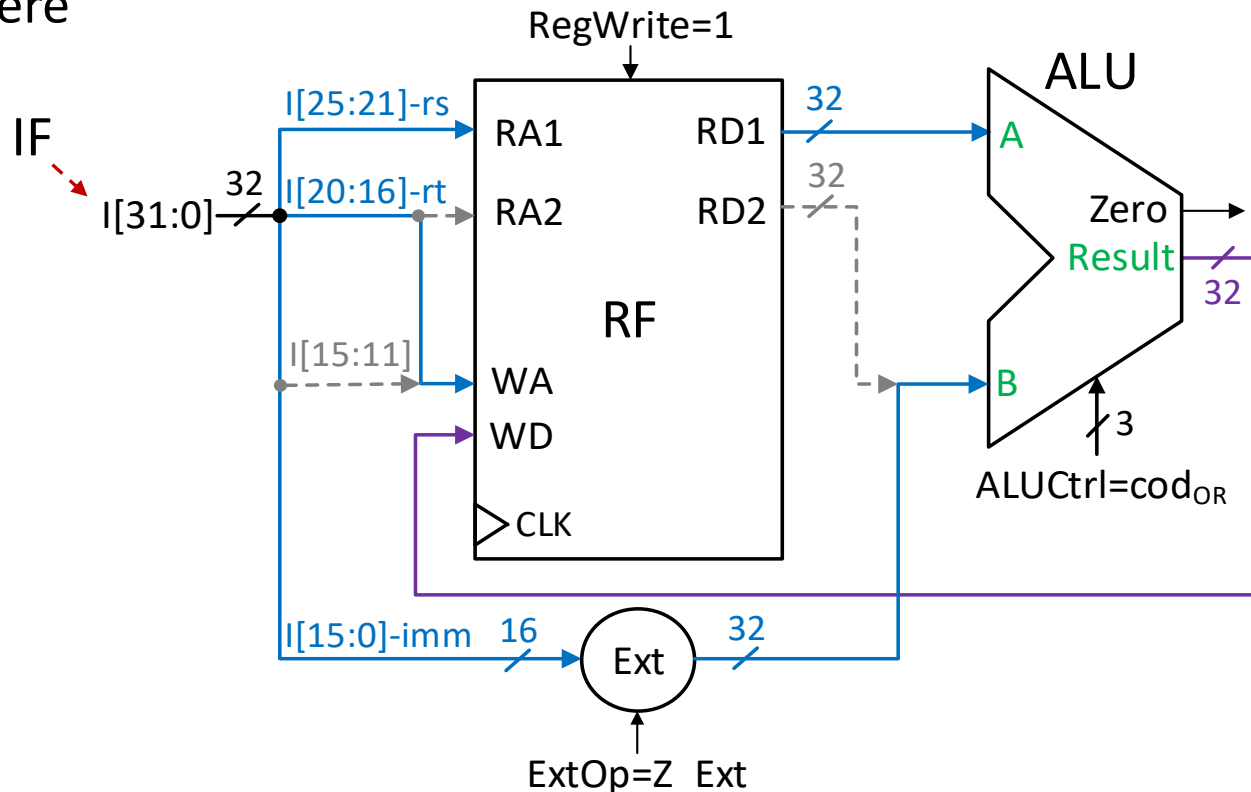


## 4.5 Asamblarea componentelor

Tip I: **ORI**:  $RF[rt] \leftarrow RF[rs] \mid Z\_Ext(imm)$

Fazele instrucțiunii (*la ciclu unic nu se referă la cicluri de ceas*):

- RA1 ← rs; WA ← rt  
A ← RD1 ← RF[RA1]; B ← Z\_Ext(imm) } Operand Fetch (OF)  
} încărcarea operanzilor
- Result ← A | B – Execute (EX), ALU Ctrl indică SAU logic
- RF[WА] ← WD ← Result – Write-Back (WB), datele sunt pregătite pentru scriere

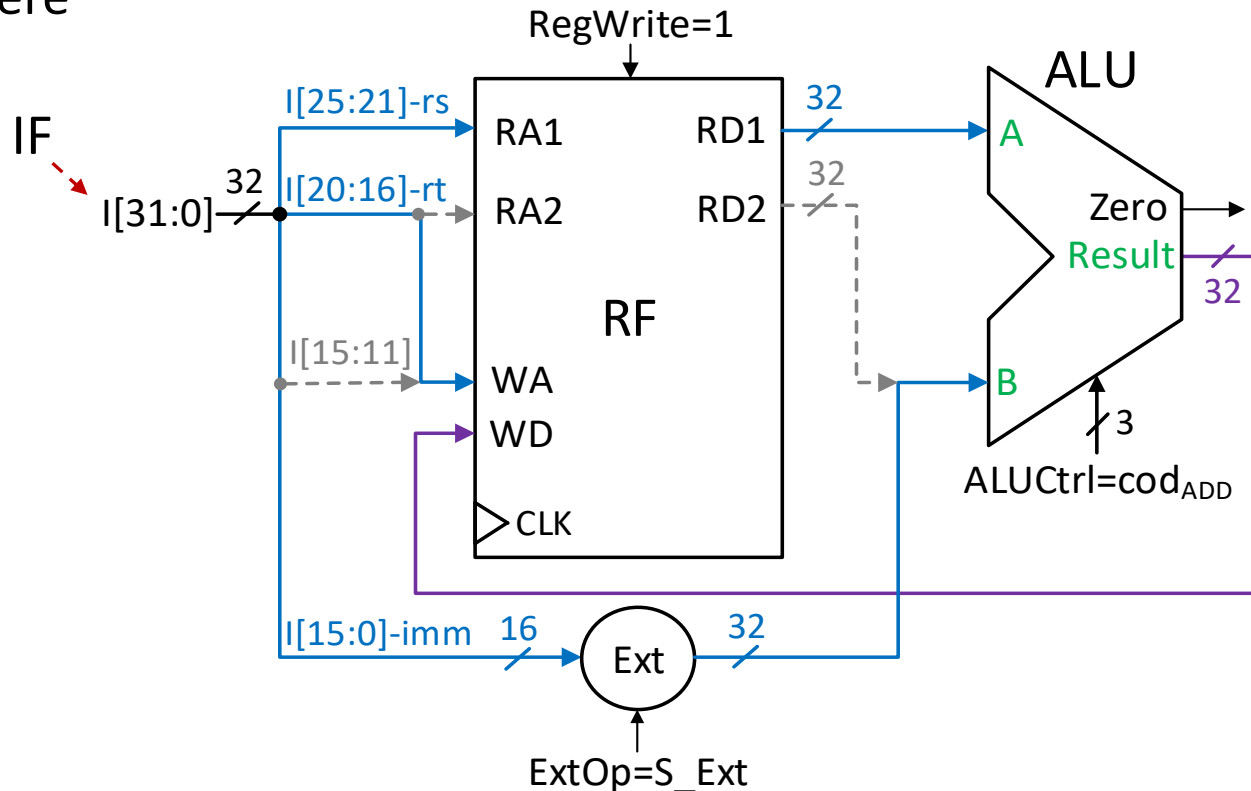


## 4.5 Asamblarea componentelor

Tip I: **ADDI**:  $RF[rt] \leftarrow RF[rs] + S\_Ext(imm)$

Fazele instrucțiunii (*la ciclu unic nu se referă la cicluri de ceas*):

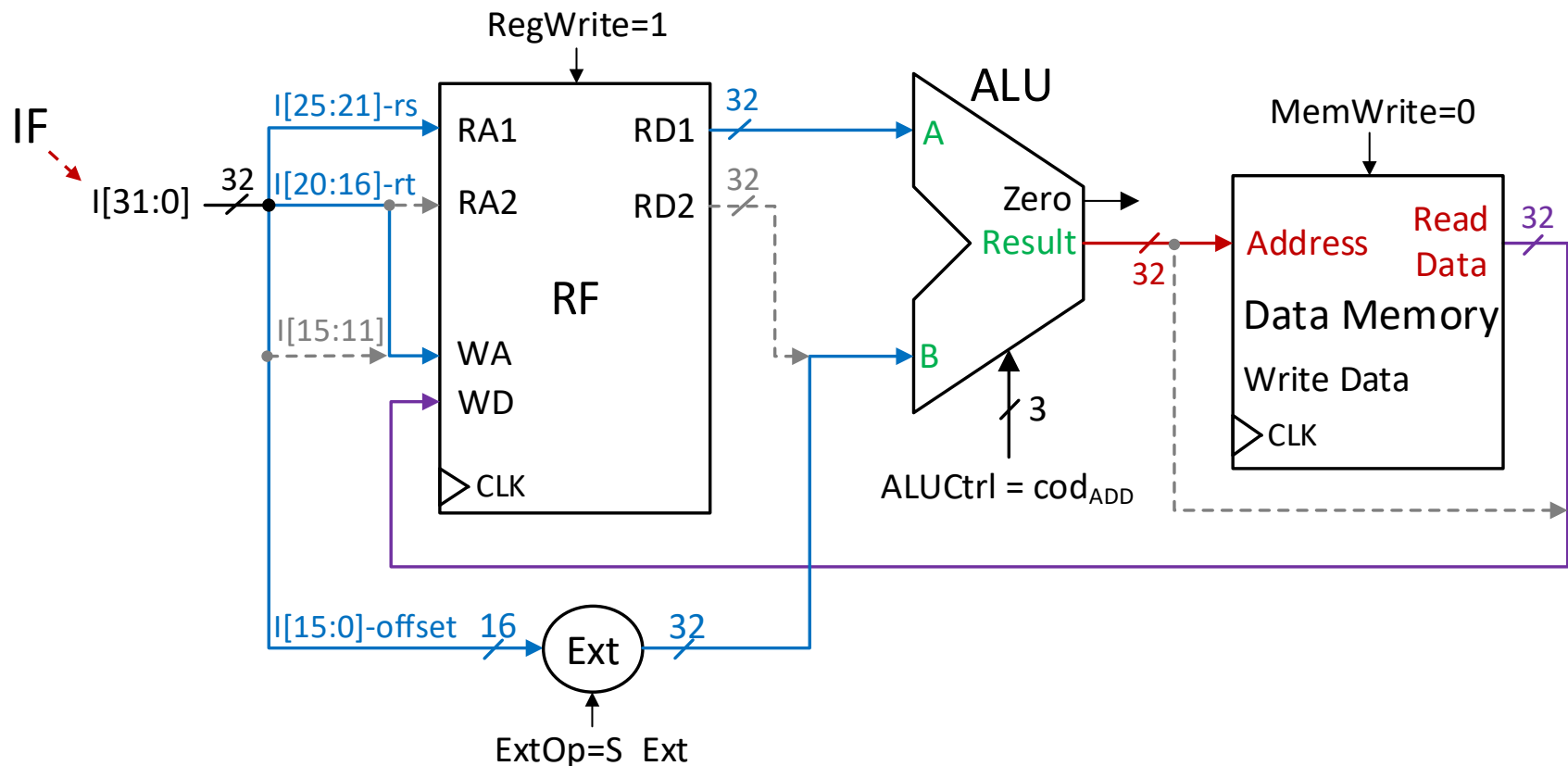
- $RA1 \leftarrow rs; WA \leftarrow rt$   
 $A \leftarrow RD1 \leftarrow RF[RA1]; B \leftarrow S\_Ext(imm)$  } Operand Fetch (OF)  
încărcarea operanzilor
- $Result \leftarrow A + B$  – Execute (EX), ALUCtrl indică adunare
- $RF[WA] \leftarrow WD \leftarrow Result$  – Write-Back (WB), datele sunt pregătite pentru scriere



Tip I: **LW**:  $RF[rt] \leftarrow Mem[RF[rs] + S\_Ext(offset)]$

Fazele instrucțiunii (*la ciclu unic nu se referă la cicluri de ceas*):

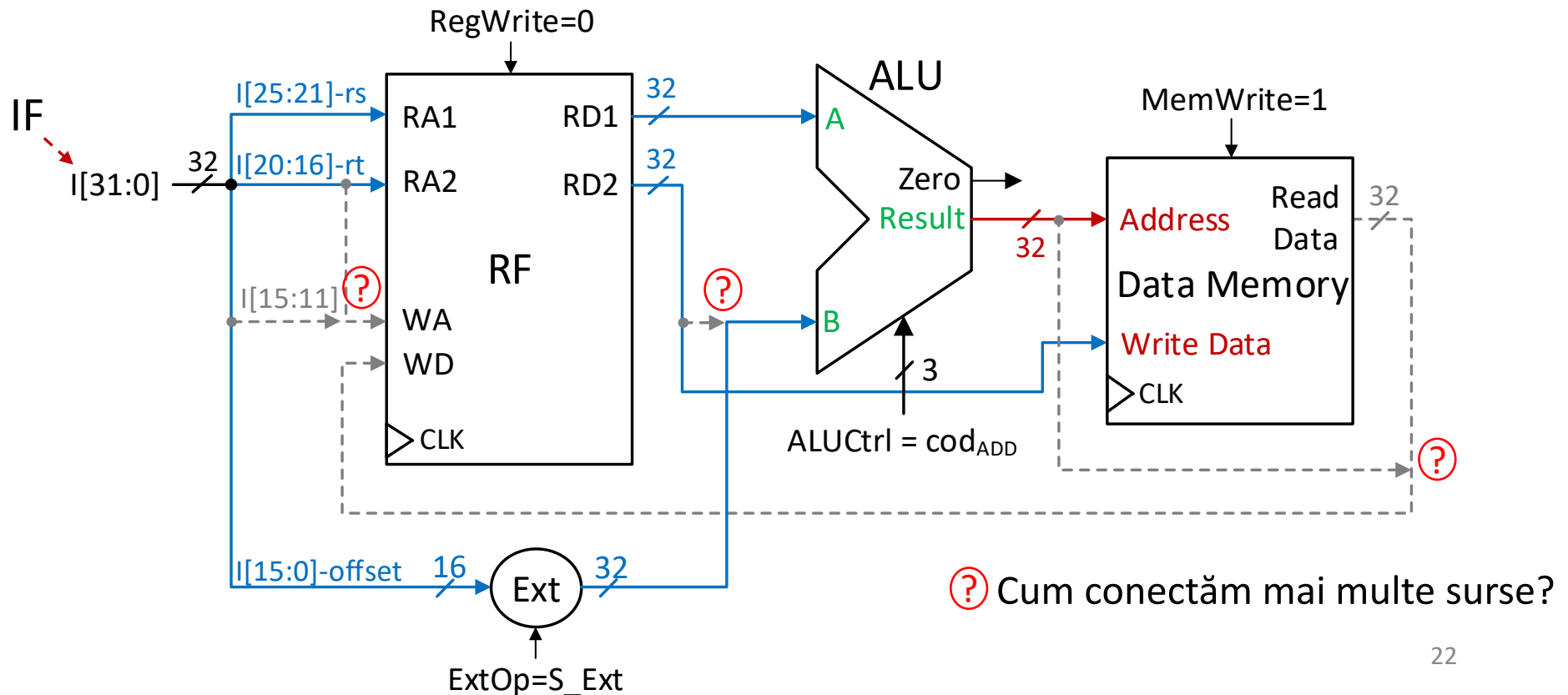
1.  $RA1 \leftarrow rs; WA \leftarrow rt$   
 $A \leftarrow RD1 \leftarrow RF[RA1]; B \leftarrow S\_Ext(offset)$  } OF
2.  $Result \leftarrow RD1 + RD2 - EX$ , ALUCtrl indică adunare
3.  $Address \leftarrow Result; Read Data \leftarrow Mem[Address] - Memory (MEM)$
4.  $RF[WA] \leftarrow WD \leftarrow Read Data - WB$ , datele sunt pregătite pentru scriere



Tip I: **SW**:  $\text{Mem}[\text{RF}[\text{rs}] + \text{S\_Ext}(\text{offset})] \leftarrow \text{RF}[\text{rt}]$

Fazele instrucțiunii (*la ciclu unic nu se referă la cicluri de ceas*):

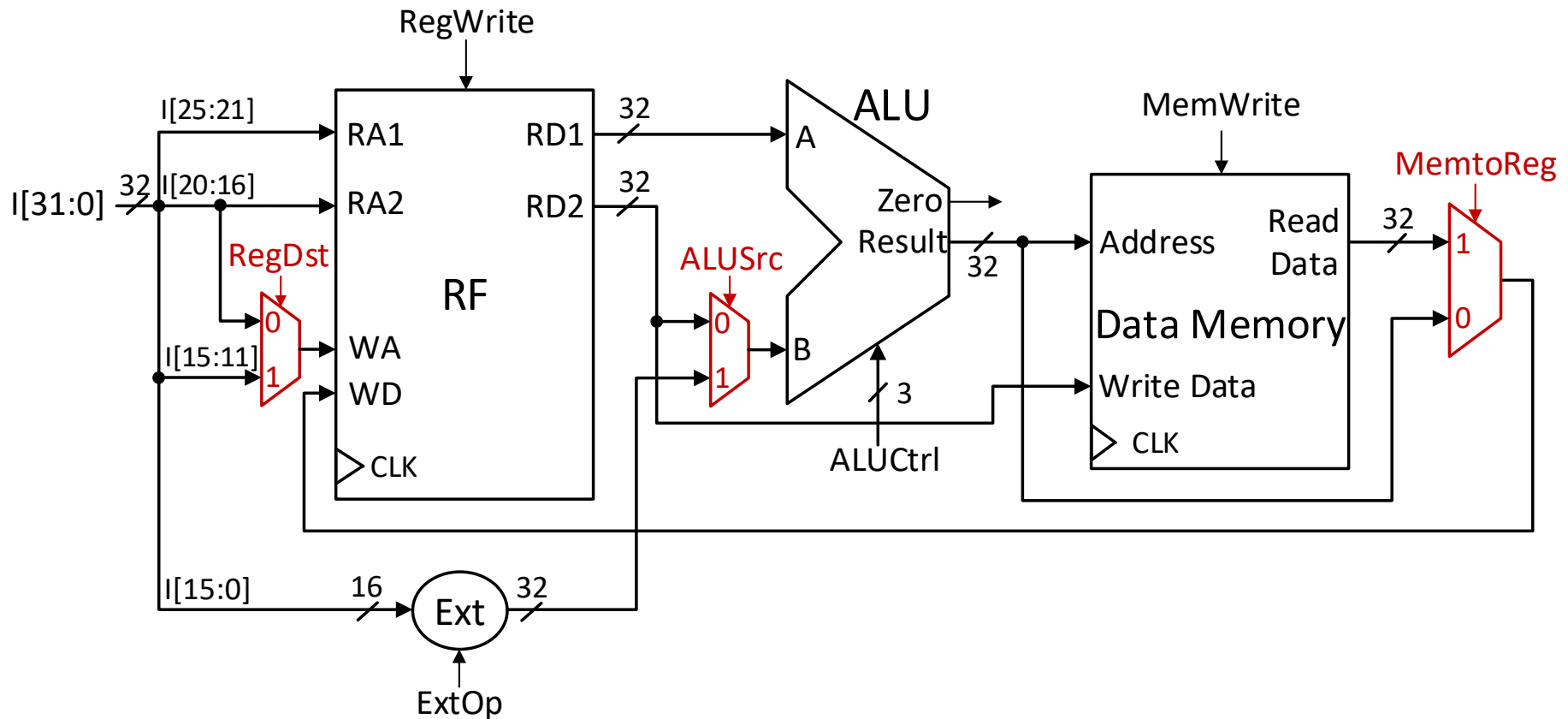
1.  $\text{RA1} \leftarrow \text{rs}; \text{RA2} \leftarrow \text{rt}$   
 $\text{A} \leftarrow \text{RD1} \leftarrow \text{RF}[\text{RA1}]; \text{B} \leftarrow \text{S\_Ext}(\text{offset}); \text{Write Data} \leftarrow \text{RD2} \leftarrow \text{RF}[\text{RA2}]$  } OF
2.  $\text{Result} \leftarrow \text{RD1} + \text{RD2} - \text{EX}$ , ALUctrl indică adunare
3.  $\text{Address} \leftarrow \text{Result}; \text{Mem}[\text{Address}] \leftarrow \text{Write Data} - \text{MEM}$
4. Nu are loc WB, deoarece nu se scrie în RF



# 4.5 Asamblarea componentelor

## Unificarea căilor de date

- Instrucțiuni acoperite: TIP R + ADDI + ORI + SW + LW.
- Se adaugă MUX-uri cu semnalele de selecție **RegDst**, **ALUSrc**, **MemtoReg**.

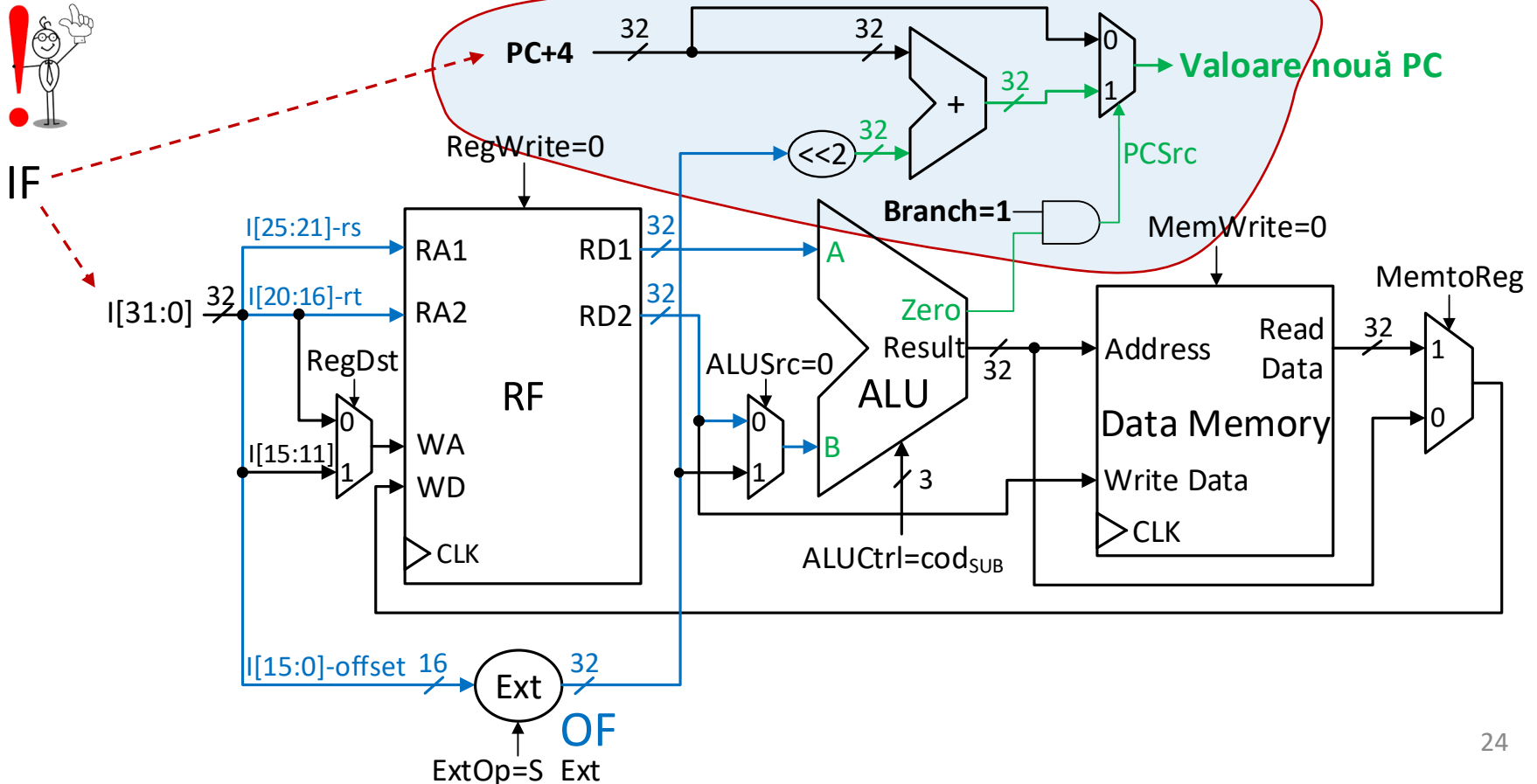


## 4.5 Asamblarea componentelor

Tip I: **BEQ**: if  $RF[rs]=RF[rt]$  then  $PC \leftarrow (PC+4) + S\_Ext(offset) \ll 2$   
else  $PC \leftarrow PC+4$

Se adaugă MUX cu selecția **PCSrc** și unități pentru calculul adresei de branch.

**Notă: La BEQ, Branch=1**

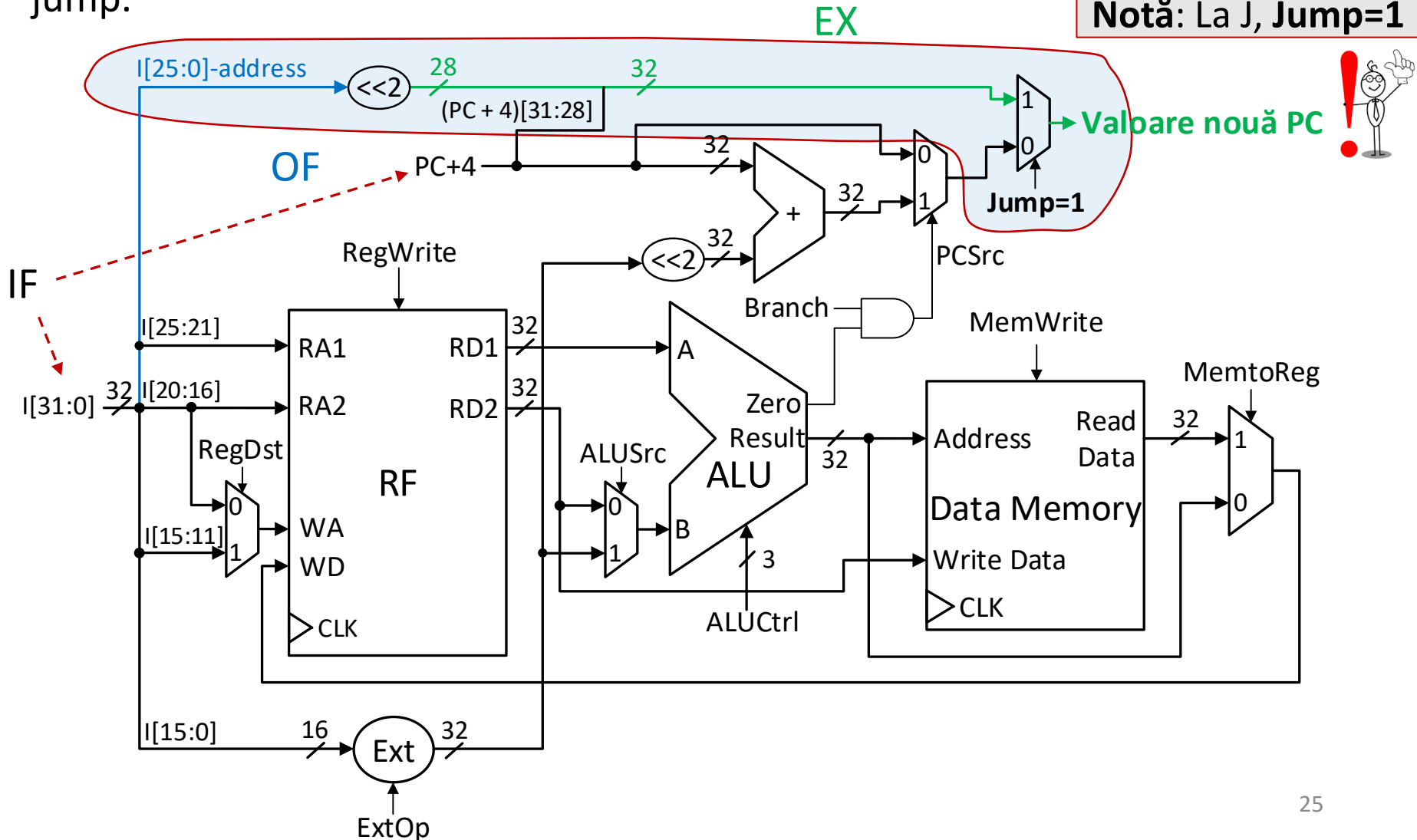


# 4.5 Asamblarea componentelor

Tip J:  $J: PC \leftarrow (PC+4)[31:28] || \text{address} || 00$

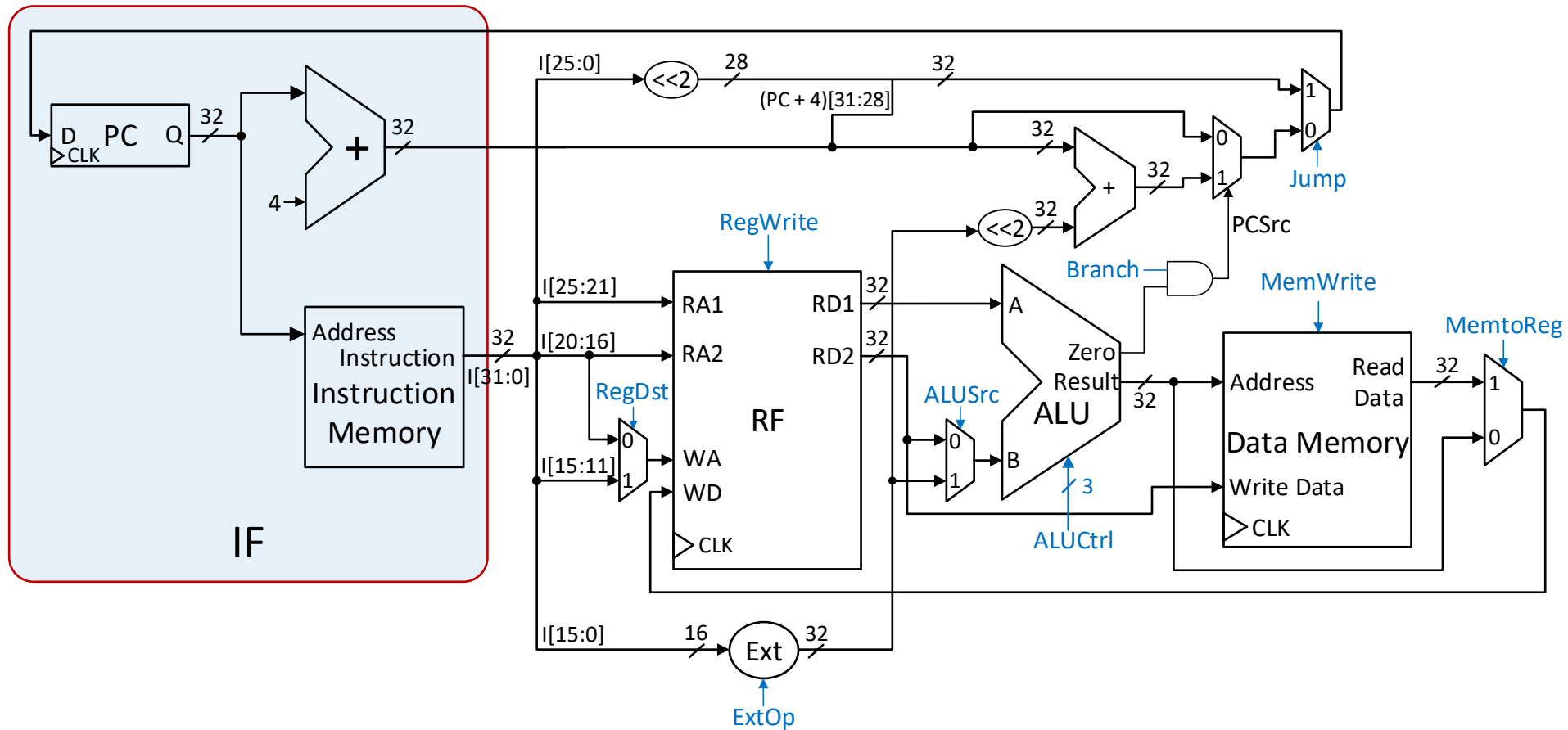
Se adaugă MUX cu selecția **Jump** și componentele pentru calculul adresei de jump.

**Notă: La J, Jump=1**



# 4.5 Asamblarea componentelor

## Unificarea IF cu restul etapelor



# 4.6 Identificarea semnalelor de control

## Pas<sub>1</sub>: Stabilirea semnificației semnalelor de control

1. **RegDst** – MUX-ul de la adresa de scriere WA a RF. Dacă are valoarea 0, atunci adresa e dată de câmpul *rt*. Dacă are valoarea 1, atunci se folosește câmpul *rd*.
2. **RegWrite** – Comanda de scriere în blocul de registre RF.
3. **ExtOp** – Controlează funcționarea unității Ext. Dacă are valoarea 0, extinde fără semn, cu 0. Dacă are valoarea 1, extinde cu semn.
4. **ALUSrc** – MUX-ul de la intrarea B în ALU. Dacă are valoarea 0, atunci pe B ajunge RD2. Dacă are valoarea 1, va ajunge imediatul extins.
5. **Branch** – Semnal activat numai în prezența instrucțiunii BEQ.
6. **Jump** – MUX-ul de salt necondiționat. Dacă are valoarea 0, spre PC trece ieșirea MUX-ului precedent. Dacă are valoarea 1, spre PC trece adresa de jump.
7. **MemWrite** – Comanda de scriere în memoria de date.
8. **MementoReg** – MUX-ul prin care se scrie înapoi rezultatul. Dacă e 0, atunci trece rezultatul de la ALU. Dacă e 1, atunci trece valoarea din memoria de date.
9. **ALUCtrl<sub>2:0</sub>** – Semnal pe 3 biți care codifică operația ALU:
  - 000 → adunare (+);
  - 100 → scădere (-);
  - 001 → ȘI logic (&);
  - 010 → SAU logic (|).



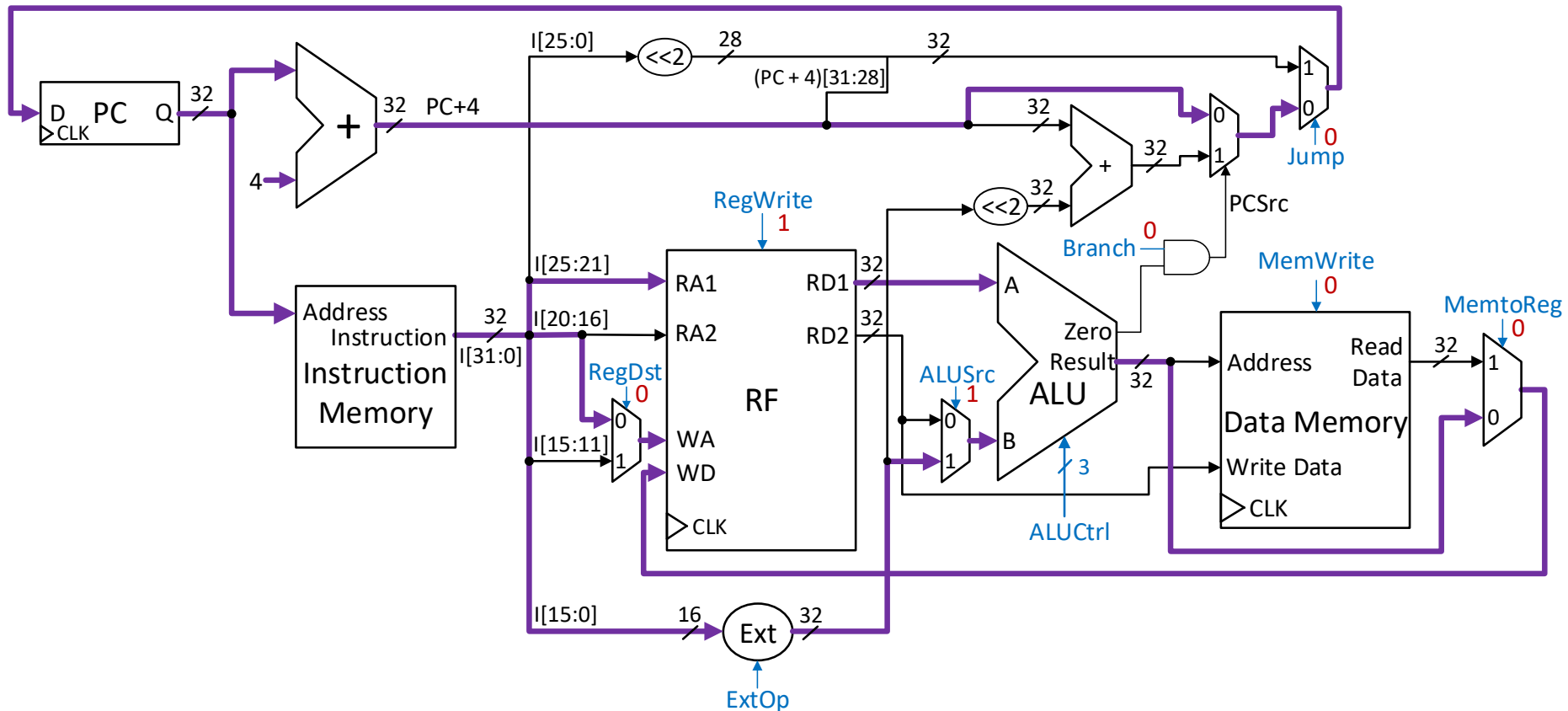
Alocarea codurilor ALUCtrl la operații se va studia în cursul următor.

**Observație:** Instrucțiunile de tip R au semnale identice cu excepția ALU Ctrl.

# 4.6 Identificarea semnalelor de control

## Pas<sub>2</sub>: Stabilirea valorilor semnalelor de control (cont)

ORI:  $RF[rt] \leftarrow RF[rs] \mid Z\_Ext(imm)$     ADDI:  $RF[rt] \leftarrow RF[rs] + S\_Ext(imm), \quad PC \leftarrow PC + 4$



| Instrucțiune | RegDst | RegWrite | ALUSrc | ExtOp | ALU Ctrl <sub>2:0</sub> | MemWrite | MemtoReg | Branch          | Jump |
|--------------|--------|----------|--------|-------|-------------------------|----------|----------|-----------------|------|
| ORI (tip I)  | 0      | 1        | 1      | 0     | 010 ( )                 | 0        | 0        | 0 <sub>29</sub> | 0    |
| ADDI (tip I) | 0      | 1        | 1      | 1     | 000 (+)                 | 0        | 0        | 0               | 0    |

# 4.6 Identificarea semnalelor de control

## Pas<sub>2</sub>: Stabilirea valorilor semnalelor de control (cont)

| Instrucțiune | RegDst | RegWrite | ALUSrc | ExtOp | ALUCtrl <sub>2:0</sub> | MemWrite | MemtoReg | Branch | Jump |
|--------------|--------|----------|--------|-------|------------------------|----------|----------|--------|------|
| ADD (tip R)  | 1      | 1        | 0      | x     | 000 (+)                | 0        | 0        | 0      | 0    |
| SUB (tip R)  | 1      | 1        | 0      | x     | 100 (-)                | 0        | 0        | 0      | 0    |
| OR (tip R)   | 1      | 1        | 0      | x     | 010 ( )                | 0        | 0        | 0      | 0    |
| AND (tip R)  | 1      | 1        | 0      | x     | 001 (&)                | 0        | 0        | 0      | 0    |
| ORI (tip I)  | 0      | 1        | 1      | 0     | 010 ( )                | 0        | 0        | 0      | 0    |
| ADDI (tip I) | 0      | 1        | 1      | 1     | 000 (+)                | 0        | 0        | 0      | 0    |
| LW (tip I)   | 0      | 1        | 1      | 1     | 000 (+)                | 0        | 1        | 0      | 0    |
| SW (tip I)   | x      | 0        | 1      | 1     | 000 (+)                | 1        | x        | 0      | 0    |
| BEQ (tip I)  | x      | 0        | 0      | 1     | 100 (-)                | 0        | x        | 1      | 0    |
| J (tip J)    | x      | 0        | x      | x     | xxx                    | 0        | x        | x      | 1    |

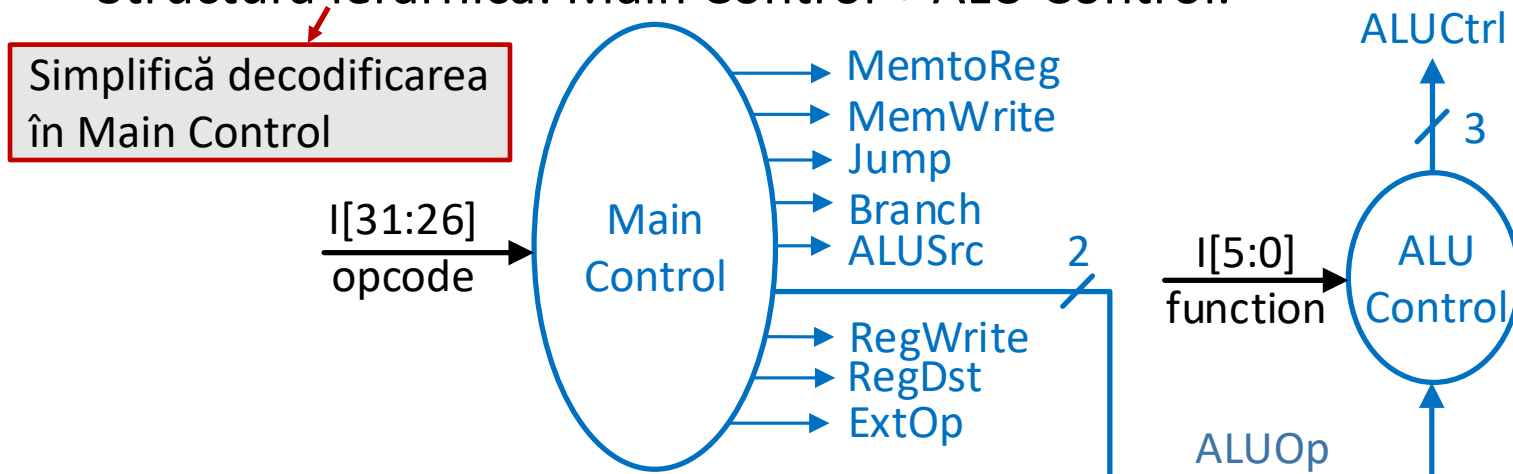


Important: De verificat acasă!

## 4.7 Implementarea logicii de control

### Unitatea de control

- Implementare combinațională;
- Structură ierarhică: Main Control + ALU Control.



### Semnificațiile codurilor ALUOp pentru unitatea ALU Control

| Instrucțiune | ALUOp <sub>1:0</sub> | Semnificația comenzii pentru ALU Control |
|--------------|----------------------|------------------------------------------|
| Tip R        | 10                   | folosește <i>function</i>                |
| ORI          | 11                   | operație SAU ( )                         |
| ADDI, LW, SW | 00                   | operație +                               |
| BEQ          | 01                   | operație -                               |
| J            | xx                   | nu contează                              |

Se alocă oricare din codurile anterioare.



## 4.7 Implementarea logicii de control

**Main Control.** Generează toate semnalele de control și  $ALUOp_{1:0}$ .

| Opcode         | RegDst | RegWrite | ALUSrc | ExtOp | $ALUOp_{1:0}$ | MemWrite | MemtoReg | Branch | Jump |
|----------------|--------|----------|--------|-------|---------------|----------|----------|--------|------|
| 000000 (tip R) | 1      | 1        | 0      | x     | 10 (R)        | 0        | 0        | 0      | 0    |
| 001101 (ORI)   | 0      | 1        | 1      | 0     | 11 ( )        | 0        | 0        | 0      | 0    |
| 001000 (ADDI)  | 0      | 1        | 1      | 1     | 00 (+)        | 0        | 0        | 0      | 0    |
| 100011 (LW)    | 0      | 1        | 1      | 1     | 00 (+)        | 0        | 1        | 0      | 0    |
| 101011 (SW)    | x      | 0        | 1      | 1     | 00 (+)        | 1        | x        | 0      | 0    |
| 000100 (BEQ)   | x      | 0        | 0      | 1     | 01 (-)        | 0        | x        | 1      | 0    |
| 000010 (J)     | x      | 0        | x      | x     | xx            | 0        | x        | x      | 1    |

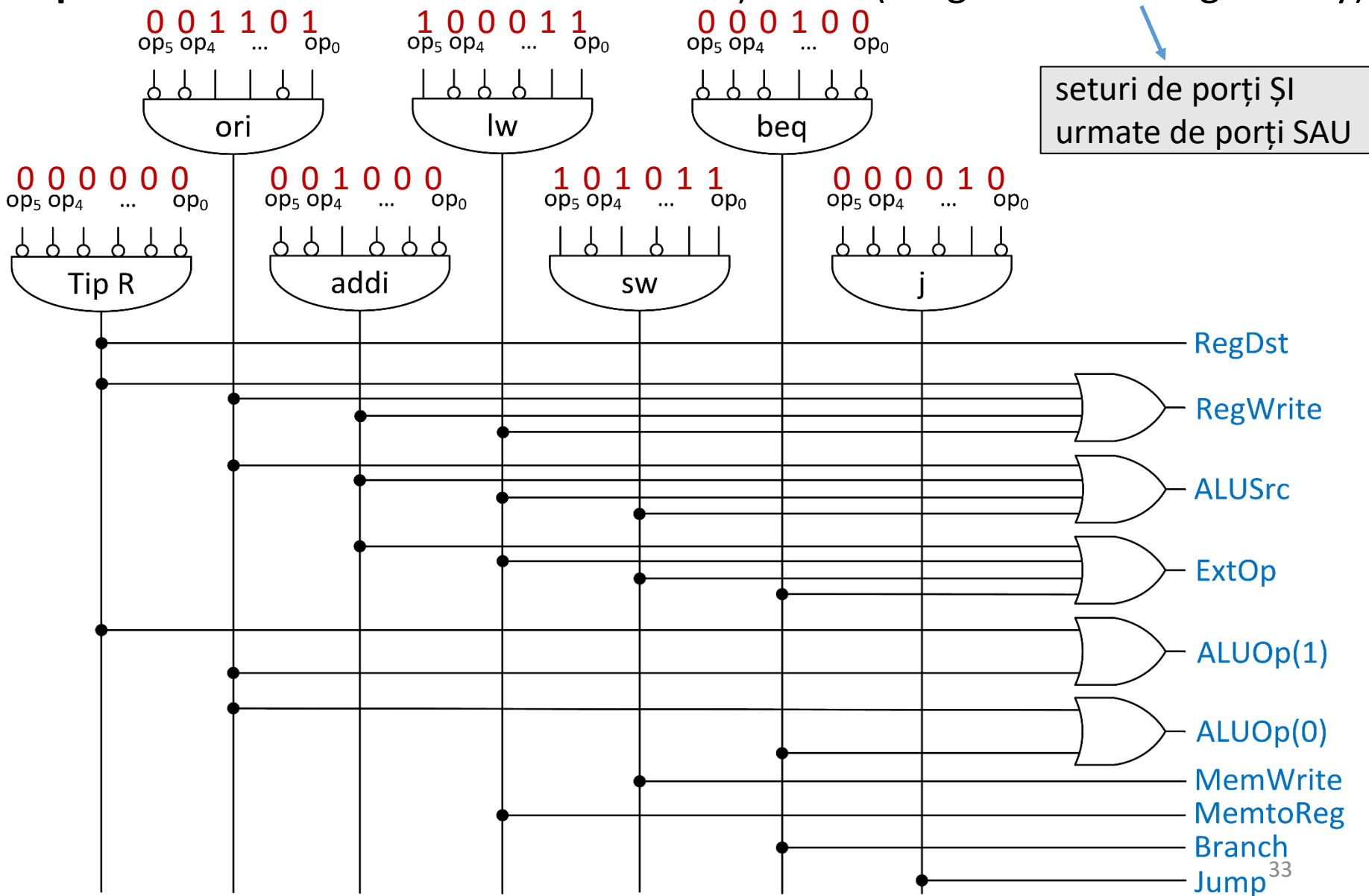
**ALU Control.** Generează valorile pentru  $ALUCtrl_{2:0}$ .

| Instrucțiune           | $ALUOp_{1:0}$                         | function | $ALUCtrl_{2:0}$ |
|------------------------|---------------------------------------|----------|-----------------|
| ADD (tip R)            | 10 (R)                                | 100000   | 000 (+)         |
| SUB (tip R)            | 10 (R)                                | 100010   | 100 (-)         |
| OR (tip R)             | 10 (R)                                | 100101   | 010 ( )         |
| AND (tip R)            | 10 (R)                                | 100100   | 001 (&)         |
| ORI (tip I)            | 11 ( )                                | xxxxxx   | 010 ( )         |
| ADDI / LW / SW (tip I) | 00 (+)                                | xxxxxx   | 000 (+)         |
| BEQ (tip I)            | 01 (-)                                | xxxxxx   | 100 (-)         |
| J (tip J)              | înlocuit de o instrucțiune de mai sus |          |                 |

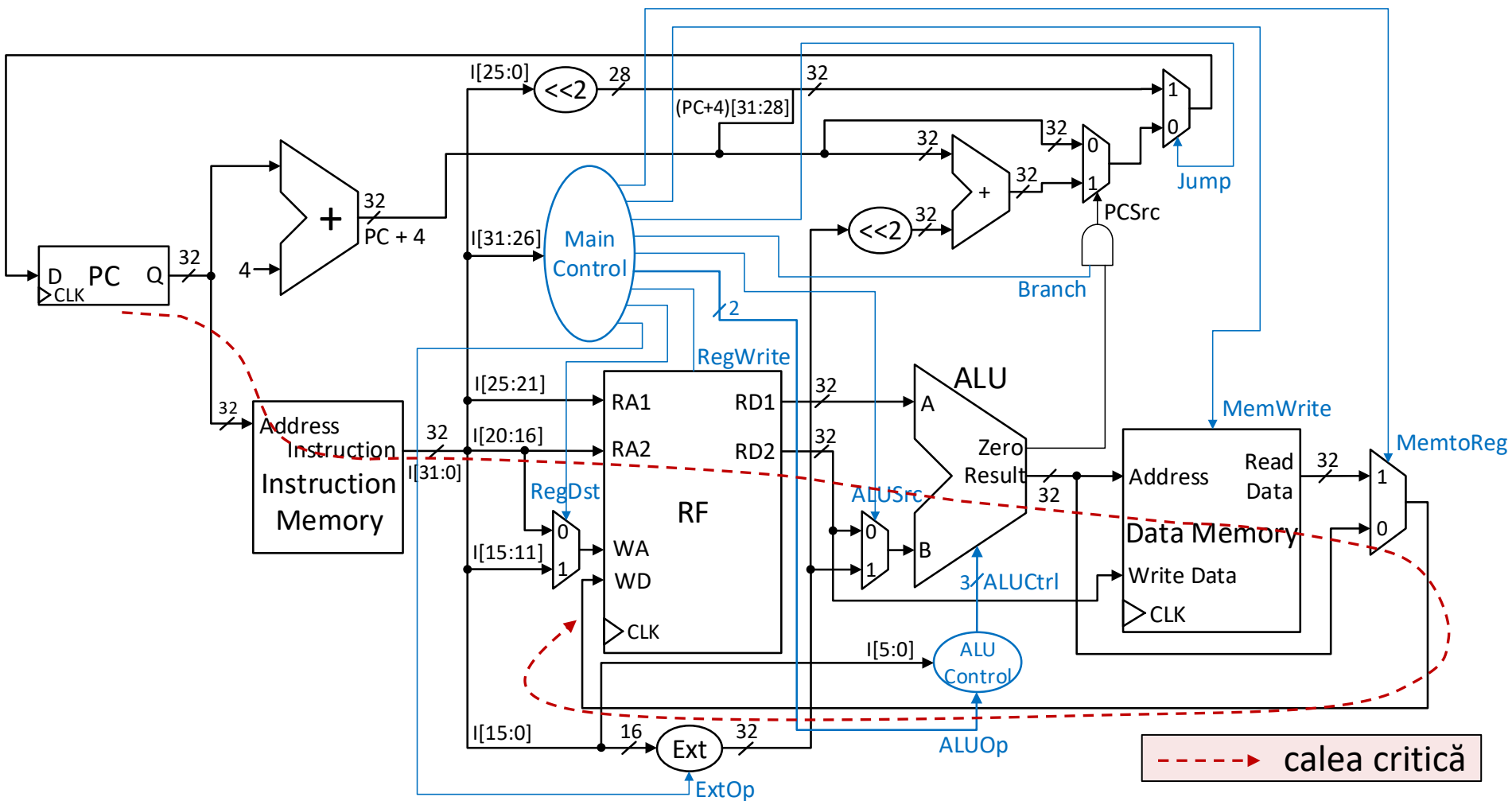
Se alocă oricare din codurile anterioare.

# 4.7 Implementarea logicii de control

**Implementarea Main Control** – se folosește PLA (Programmable Logic Array)



# Calea de date + Unitățile de control



**Dezavantaj ciclu unic:** Perioada de ceas este limitată de calea critică (**instrucțiunea LW**).

- $T_{CLK} = T_{citire} \text{ PC după front} + T_{acces} \text{ la Memoria de Instrucțiuni} + T_{acces} \text{ la RF} +$   
 $+ T_{propagare} \text{ MUX}_{ALUSrc} + T_{propagare} \text{ ALU} + T_{acces} \text{ la Memoria de Date} +$   
 $+ T_{propagare} \text{ MUX}_{MemtoReg} + T_{setup} \text{ scriere în RF}$

# Concluzii

Perioada de ceas necesară pe categorii de instrucțiuni:

|                   |                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------|
| Aritm. și logice: | $T_{PC} + T_{InstrMem} + T_{RF} + T_{muxALUSrc} + T_{ALU} + T_{muxMemtoReg} + T_{setup}$               |
| LW:               | $T_{PC} + T_{InstrMem} + T_{RF} + T_{muxALUSrc} + T_{ALU} + T_{DataMem} + T_{muxMemtoReg} + T_{setup}$ |
| SW:               | $T_{PC} + T_{InstrMem} + T_{RF} + T_{muxALUSrc} + T_{ALU} + T_{DataMem} + T_{setup}$                   |
| Branch:           | $T_{PC} + T_{InstrMem} + T_{RF} + T_{muxALUSrc} + T_{ALU} + T_{muxPCSrc} + T_{muxJump} + T_{setup}$    |
| Jump:             | $T_{PC} + T_{InstrMem} + T_{muxPCSrc} + T_{muxJump} + T_{setup}$                                       |

$$T_{imp_{program}} = N_{r_{instrucțiuni}} \cdot \frac{\text{cicluri}}{\text{instrucțiune}} \cdot T_{\text{ciclu mare}}$$

1

**Obs:** Durata LW mai mare ca la alte instrucțiuni => timp pierdut prin inactivitate.

# Probleme propuse

Implementarea de instrucțiuni:

1. Tipice: ADD, SUB, AND, OR, LW, SW, BEQ, J, ANDI, ORI, SLL, SRL, SRA.

2. Altele:

- BLTZ, BNE – variante de branch (salturi condiționate).
- JR – Jump Register:  $JR \$rs \Rightarrow PC \leftarrow \$rs$ . Indiciu: Se implementează de tip I.
- LW – fără offset:  $LW \$rt, \$rs \Rightarrow \$rt \leftarrow Mem[\$rs]$ .
- JAL – Jump And Link:  $JAL address \Rightarrow J address; RF[31] \leftarrow PC+4$ . Indiciu: Pentru **link** se realizează cu un nou MUX o conexiune suplimentară de la PC+4 la intrarea WD a RF, iar pe WA se pune 31 prin adăugarea altui MUX. Pentru **salt** se activează [Jump](#).
- JM – Jump Memory:  $JM offset(\$rs) \Rightarrow PC \leftarrow Mem[\$rs + S\_Ext(offset)]$ .
- ADDM – ADD Memory:  $ADDM \$rt, offset(\$rs) \Rightarrow \$rt \leftarrow \$rt + Mem[\$rs + S\_Ext(offset)]$ .
- WAI – Where Am I:  $WAI \$rt \Rightarrow \$rt \leftarrow PC$ .

## Referințe

[1] F. Oniga, “De la bit la procesor. Introducere în arhitectura calculatoarelor”, Cap. 9.3, 9.4, Ed. U.T. Press, 2019, ISBN 978-606-737-366-0.

[2] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 4.1-4.4, Anexa B8, D, Ed. Morgan–Kaufmann, 2013.

[3] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.

[4] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.